ZP-E Illustrated Companion

Four frameworks, one event — the causality axiom is retired, and the shape of the snap becomes a theorem

Bridge Document | DA-1 / T-SNAP Update | "One event" is the framework's reading, not a theorem

ZP Companion | Initial: April 2026 | Current: 1.34, November 2026

This companion explains the ideas in plain language with diagrams and real-world examples. It is not the formal document — every claim here restates a result already proved in the corresponding technical document. Consult that document for the authoritative mathematics.

What Is ZP-E Doing?

ZP-E is the cross-framework synthesis. Written last — after ZP-A through ZP-D are each internally closed — its job is to show how each of the four frameworks describes the Binary Snap from its own mathematical vantage point. Each description is proved within its own framework, from that framework's own premises (ZP-D's include its design premise DP-1). ZP-H T-H3 (th3_snap_all_functors) is their conjunction, four domain facts joined by "and", with no functor or morphism in its statement, and ZP-H identifies none of the objects each framework located; where a shared role is checked, it is checked in one framework at a time (ZP-H Section VI). Reading the four as one event is the framework's interpretation, not a theorem. The bottoms the four descriptions start from form a family (the bottom family, MC-1), not one object: an identity across the frameworks is retired as ill-typed, because their objects live in different mathematical types.

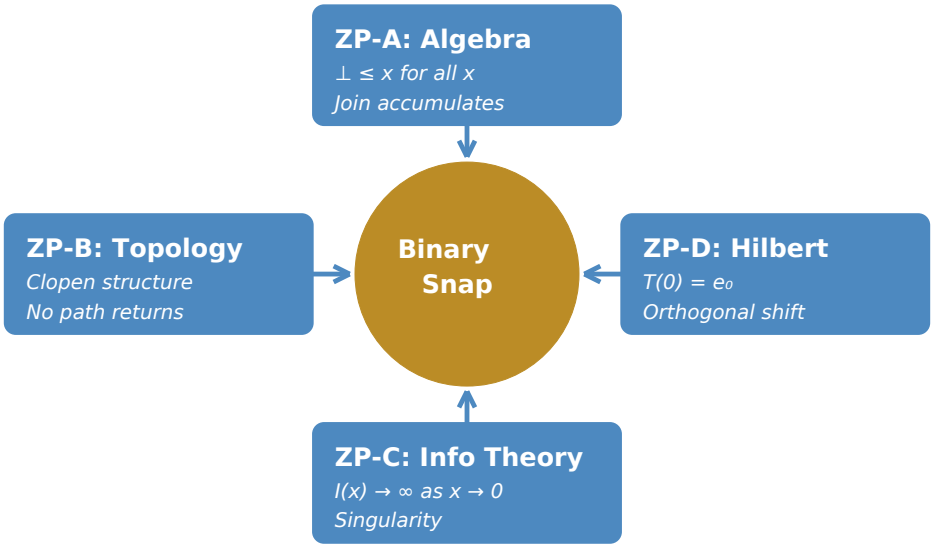
ZP-E does not re-derive anything. It imports closed results from each sub-document and shows their consistency. Where a cross-framework connection requires an assumption, that assumption is named explicitly as a bridge axiom — not hidden inside the argument.

Four Descriptions of the Binary Snap

The Binary Snap — the transition from nothing (⊥) to the first state above it (ε_0, in the discrete state chart) — looks different depending on which mathematical language you use. ZP-E's central result is that the four descriptions are consistent, each proved in its own framework. Reading them as one event seen from four angles is the framework's interpretation, not a theorem.

The symbol ε_0 has two charts, and they are not merged. In the ordinal chart, ε_0 is the proof-theoretic ordinal of Peano Arithmetic, the minimum ordinal up to which PA cannot prove transfinite induction. Goodstein's theorem is the standard witness: every Goodstein sequence eventually terminates, but PA cannot prove this (Kirby and Paris, 1982, as ZeroParadox/Ordinal/Goodstein.lean records); the proof requires transfinite induction up to ε_0. That ordinal is at once the least fixed point of the map $\alpha \mapsto \omega^\alpha$ and the supremum of the

tower ω, ω^ω, ω^(ω^ω), ..., and neither face replaces the other. In that chart the snap does land on it: closing ⊥ under $\alpha \mapsto \omega^\alpha$ gives ε_0 in one step. In the discrete state chart, ε_0 names the first state above ⊥, with nothing in between, while infinitely many ordinals lie between 0 and the ordinal ε_0. The formal ZP-E treats the shared symbol as an analogy (Remark R-ε_0) and claims no embedding of one chart into the other.



The Binary Snap (amber center) described simultaneously in all four frameworks. Each arrow represents one framework's mathematical description of the Snap; that the four describe one event is the framework's reading, not a theorem.

Real-world example — A car crash described by four witnesses

An engineer (forces), a doctor (injuries), a lawyer (liability), and a physicist (energy) each describe the same crash completely within their own discipline. ZP-E reads the four mathematical frameworks as standing in this relationship to the Binary Snap: each description is proved in its own framework, and that they describe one event is the framework's interpretation.

Remember: The car crash illustrates what it means to describe one event in multiple frameworks. The Zero Paradox is not about car crashes — the car crash is an analogy for the first transition from ⊥ to ε_0 — a transition the framework commits to, rather than one any join-semilattice compels. A join-semilattice in which T-SNAP holds and nothing moves is exhibited in the Lean source.

The Central Advance: AX-1 Retired, T-SNAP Proves the Shape

In earlier versions, the Binary Snap causality was listed as AX-1 — an axiom: a foundational assumption, taken without derivation. The claim was: when P_0 is reached, the Snap happens. Why? Because AX-1 says so. That claim bundled two things: the shape of the Snap (what a step off ⊥ looks like) and its occurrence (that the step is taken). AX-1 is now retired, and its content was split in two: T-SNAP proves the shape, and that the Snap occurs is stated separately: it follows from the occurrence commitment (instantiation occurs) together with DA-1 (closed given DP-2).

The DA-1 insert puts an argument where "Because AX-1 says so" stood, and the occurrence commitment stays in place. The argument runs as follows. The occurrence commitment says instantiation occurs as a running configuration: the configuration reaching P_0 is a running machine's current configuration, not an inert string. That is a commitment, not a theorem (formal ZP-E, T-SNAP Step 2). Given it, the machine has left c_0 and is at c_1, by the definition of the first running configuration (D7); DA-1 (closed given DP-2) uses that commitment and does not supply it. Any running configuration passes through c_1 (definition). c_1 is not $\perp$ (L-RUN). No program avoids this (TQ-IH). A state change from $\perp$ to any state other than $\perp$ is the Binary Snap (ZP-A D2). The Snap is derived, and that sentence has two true readings. The Lean proof is a fact about a fixed two-state machine type and takes no assumptions: it fixes the shape of the step, not that the step happens. The argument about the framework's own sequence uses commitments, among them AX-B1 (existence is binary: a state exists or it does not) and CC-1 (the sequence starts at $\perp$); that the step actually happens rests further on DA-1, which is closed only given DP-2, and on the framework's commitment that instantiation occurs. The formal ZP-E states these premises under Premises of T-SNAP, and the Lean t_snap_given takes two of them as its hypotheses: the start at $\perp$ and the first step being taken.

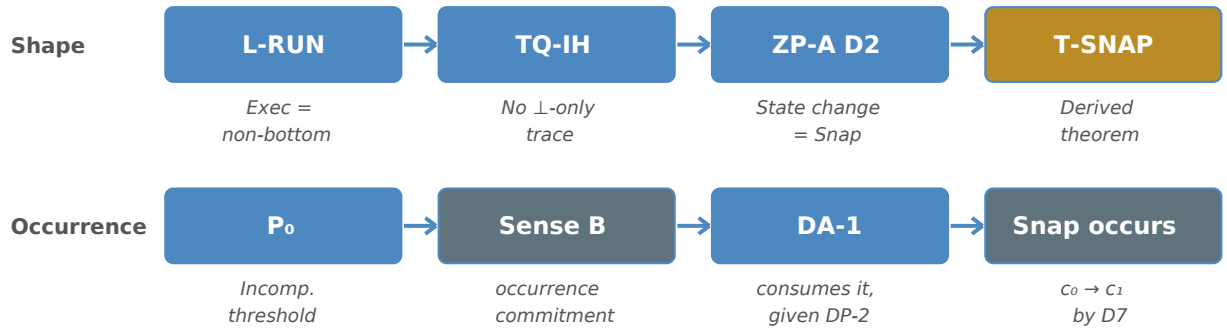
DA-1 is a Derived Proposition, closed given DP-2, as the next paragraph explains. That a configuration at P_0 is executing is not part of what it derives: that is DP-2's precondition, and it is what the occurrence commitment asserts. ZP-A CC-2, the Quine atom, motivates it. In the die it is a role, a point that steps to itself and to nothing else; under a self-application map supplied by AbstractSelfApp, $\perp$ of a ZP-A semilattice is its only occupant by that class's own fields (selfApp_quine_role_iff), which the constant map onto $\perp$ also meets, so the content is in the host. That any host of the role is not well-founded is forced (quineAtomHost_not_wellFounded); which requirements to demand of it, and adopting AFA, stay commitments (argued, not proved). The bottom element $\perp$ is a Quine atom, a self-containing object with no external position from which it could be interpreted as a static description. So if $\perp$ executes at all, nothing outside it can be the executor; the executor is $\perp$ itself. That rules out an external executor, not an inert $\perp$: the two-state machine type in the Lean source has a Quine-atom bottom and also admits a dynamics in which nothing moves. That the configuration reaching P_0 is running is the occurrence commitment, not a consequence of the Quine atom (CC-2).

The two-layer structure of DA-1: DA-1 rests on two explicit layers. The first is the formal conditional: DP-2 (Execution Distinguishability) establishes that machine states carry execution history independently of output values. From DP-2, Lean 4 can derive da1_minimal_path — a proof that the configurations before and after instantiation produce the same output value (c_0) while their machine states differ (c_0 before, c_1 after). It takes no assumption and does not say the step is taken; that comes from the precondition below. `#print axioms` confirms zero axiom dependencies. This is the first Lean formalization of the core DA-1 claim, not just the surrounding algebra. The second layer asks: is DP-2's precondition met, that is, is the configuration reaching P_0 running rather than inert? That precondition is what the occurrence commitment asserts. Three arguments bear on it, and none derives it.

Three arguments for the precondition (ZP-E's Paths 1, 2 and 3): (1) CC-2/R3 (ZP-A): $\perp$ is the Quine atom (CC-2) — it interprets itself, leaving no external position from which it could be read as a static description. As shown above, that rules out an external executor, not an inert $\perp$. (2) Informational, in two charts: surprisal grows without bound with depth in the ball hierarchy, each value finite (ZP-C L-INF, l_inf); and at 0 of $\mathbb{Q}_2$, the one point in every ball of the chain centred at 0, the 2-adic additive valuation is $\top$ (padic_addVal_bot). $\perp$ of the machine-state type, c_0, corresponds to that point. From this the framework argues that no finite static distribution can represent $\perp$. That rules out a finite static description; the step from there to executing is a bridge principle of its own, a missing principle and not a missing proof, which

the framework does not adopt and which is separate from the occurrence commitment. (3) Computational (incompressibility as self-description): at the incompressibility threshold P_0, the configuration c_1 is incompressible ($K(c_1|n) \geq |c_1|$, the ratio $K(c_1|n)/|c_1|$ being 1 up to an additive constant in K), so no shorter external program generates it. That rules out a shorter external generator, not an inert string: an incompressible string written down and never run has no shorter generator and is still not executing. So Path 3 does not derive executing from incompressibility; that the configuration reaching P_0 is running, not inert, is what the occurrence commitment asserts. All three share D7's static/executing dichotomy as background and none is circular with DP-2.

What Lean checks (ZP-K): ZP-K adds a KleeneStructure instance for MachinePhase (machinePhaseKleene). Path 1's Lean counterpart (AFA self-containment) is da1_closed_concrete, which proves IsQuineAtom(⊥ : MachinePhase) and nothing computational; Path 3's Lean counterpart is the machinePhaseKleene instance's botCode_is_quine field, a KleeneStructure requirement that constant codes also meet, not a witness of execution and not a second independent proof. The two are carried together by da1_paths_unified as a conjunction; that they name one structural fact is the framework's reading. Neither Path 2 (unbounded surprisal) nor Path 3 (incompressibility) derives executing: the step from unbounded surprisal to executing is a bridge principle of its own, a missing principle that the framework does not adopt and that is separate from the occurrence commitment; for Path 3 the reason is item (3) above, a fact of algorithmic information theory and not among what Lean checks. The formal grounding of DA-1 is therefore DP-2, through da1_minimal_path.



The T-SNAP derivation chain. The chain uses commitments, among them AX-B1 and CC-1 ($S_0 = \perp$), and fixes the shape of the Snap; that the Snap occurs rests further on the occurrence commitment (instantiation occurs), whose content is DA-1's precondition, and on DA-1 (closed given DP-2), which consumes it. AX-1 is retired: its shape is now a theorem (T-SNAP, amber), and that the Snap occurs is stated separately, following from that commitment together with DA-1.

Real-world example — A legal case proved in part

A charge can bundle two claims: "the defendant was at the scene" and "the defendant acted." Surveillance footage can prove the first while the second still rests on testimony the court chooses to accept. AX-1 bundled two claims in the same way. T-SNAP proves the first, the shape of the Snap, which was once assumed. The second, that the Snap happens, was not proved along with it: it follows from the occurrence commitment (instantiation occurs) together with DA-1 (closed given DP-2).

What the Framework Still Assumes

After T-SNAP, the framework states three named axioms:

Commitment	Statement
AX-B1	Binary Existence. A state either exists or it does not. No third option. The framework's ONE substantive modelling commitment: that the outcome space is DISCRETE rather than a continuum. The two states being distinct is decidable; the choice of a discrete alphabet is not, and the real numbers are where it fails.
AX-G1	Initial Object Exists, and No Terminal Object. There is a starting point that reaches everything in exactly one way, and no end point that everything reaches in exactly one way. The starting point is not a novel commitment — it is grounded in $\perp$ as the bottom element of the ZP-A semilattice. The absence of an end point is ZP-G's own commitment: in ZP-A it is an optional hypothesis, which ZP-A's two-state carriers do not satisfy.
AX-G2	Source Asymmetry. Nothing returns to the initial object. The origin is unreachable from outside. Not a novel commitment — follows from ZP-A antisymmetry and ZP-B C3.

AX-1 is not on this list: it is retired, and its shape is Theorem T-SNAP. Further commitments are not axioms and are stated where they are used, among them the occurrence commitment (that instantiation occurs; together with DA-1 it gives that the Snap happens), CC-1 ($S_0 = \perp$, a ZP-A Conditional Claim), the choice of starting point that T-SNAP is derived given, and CC-2 (the Quine atom, a ZP-A Forced Metatheoretic Commitment: Foundation-freeness of its host is forced, the host's requirements and AFA are chosen), which motivates DA-1's precondition; DA-1 itself is closed given DP-2. The framework makes no stronger claim than it has to.

> Remember: The structural results — monotonicity, clopen separation, informational singularity, orthogonal shifts — hold in any instantiation of the framework. The framework makes no claims about which physical theory, if any, instantiates it.